

Labwork 5: Gaussian Blur Convolution

Pham Tien Nhat

October 3, 2026

1 Implementation

The 7x7 Gaussian filter is the matrix of the slide, copied to the GPU.

```
gaussian = np.array([[0, 0, 1, 2, 1, 0, 0],
                    [0, 3, 13, 22, 13, 3, 0],
                    [1, 13, 59, 97, 59, 13, 1],
                    [2, 22, 97, 159, 97, 22, 2],
                    [1, 13, 59, 97, 59, 13, 1],
                    [0, 3, 13, 22, 13, 3, 0],
                    [0, 0, 1, 2, 1, 0, 0]], np.int32)
devFilter = cuda.to_device(gaussian)
```

Each thread blurs one pixel without shared memory: it multiplies the 7×7 neighbours of its pixel by the filter, sums them and divides by 1003.

```
@cuda.jit
def blur(src, dst, filt):
    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if (tid < 3 or tidy < 3 or
        tid >= src.shape[1] - 3 or tidy >= src.shape[0] - 3):
        return
    s = 0
    for i in range(7):
        for j in range(7):
            s += src[tidy + i - 3, tid + j - 3] * filt[i, j]
    dst[tidy, tid] = np.uint8(s / 1003)
```

```
devBlur = cuda.to_device(devGray.copy_to_host())
blur[gridSize, blockSize](devGray, devBlur, devFilter)
```

The filter is copied once per block into a 7×7 shared array with shared memory, and every thread of the block reads the filter from it. `cuda.syncthreads()` makes the whole block wait until the copy is done.

```
@cuda.jit
def blurShared(src, dst, filt):
    tile = cuda.shared.array((7, 7), np.int32)
    if cuda.threadIdx.x < 7 and cuda.threadIdx.y < 7:
        tile[cuda.threadIdx.x, cuda.threadIdx.y] = \
            filt[cuda.threadIdx.x, cuda.threadIdx.y]
    cuda.syncthreads()

    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if (tid < 3 or tidy < 3 or
```

```

        tidx >= src.shape[1] - 3 or tidy >= src.shape[0] - 3):
    return
s = 0
for i in range(7):
    for j in range(7):
        s += src[tidy + i - 3, tidx + j - 3] * tile[i, j]
dst[tidy, tidx] = np.uint8(s / 1003)

```

The CPU version uses the same blur with `for` loops, and its time is the baseline of the speedup.

```

gray = devGray.copy_to_host()
blurCPU = gray.copy()
start = time.time()
for y in range(3, h - 3):
    for x in range(3, w - 3):
        s = 0
        for i in range(7):
            for j in range(7):
                s += int(gray[y + i - 3, x + j - 3]) * \
                    int(gaussian[i, j])
        blurCPU[y, x] = np.uint8(s / 1003)
cpu_time = time.time() - start

```

2 Experimental Results

4 2D block sizes are tested: (4,4), (8,8), (16,16) and (32,32), and the speedup is the CPU time divided by the GPU time, with and without shared memory.

```

block_sizes = [(4, 4), (8, 8), (16, 16), (32, 32)]
speedups = []
speedupsShared = []

for blockSize in block_sizes:
    gridSize = (math.ceil(w / blockSize[0]), math.ceil(h / blockSize[1]))

    start = time.time()
    blur[gridSize, blockSize](devGray, devBlur, devFilter)
    cuda.synchronize()
    gpu_time = time.time() - start
    speedups.append(cpu_time / gpu_time)

    start = time.time()
    blurShared[gridSize, blockSize](devGray, devBlurShared, devFilter)
    cuda.synchronize()
    gpu_time_shared = time.time() - start
    speedupsShared.append(cpu_time / gpu_time_shared)

    print(f"Block size: {blockSize}")
    print(f"Grid size: {gridSize}")
    print(f"GPU time: {gpu_time} seconds")
    print(f"GPU time (shared memory): {gpu_time_shared} seconds")

plt.figure(figsize=(8, 5))
plt.plot([str(b) for b in block_sizes], speedups, marker='o',
        label='without shared memory')
plt.plot([str(b) for b in block_sizes], speedupsShared, marker='o',
        label='with shared memory')
plt.xlabel("2D Block Size")
plt.ylabel("Speedup")
plt.title("CUDA 2D Block Size vs Speedup (Gaussian Blur)")

```

```
plt.legend()
plt.grid(True)
plt.show()
```

Figure 1 shows the relationship between the 2D block size and the speedup, where with shared memory surpassed without one at (16,16), (4,4), (32,32) at speedup.

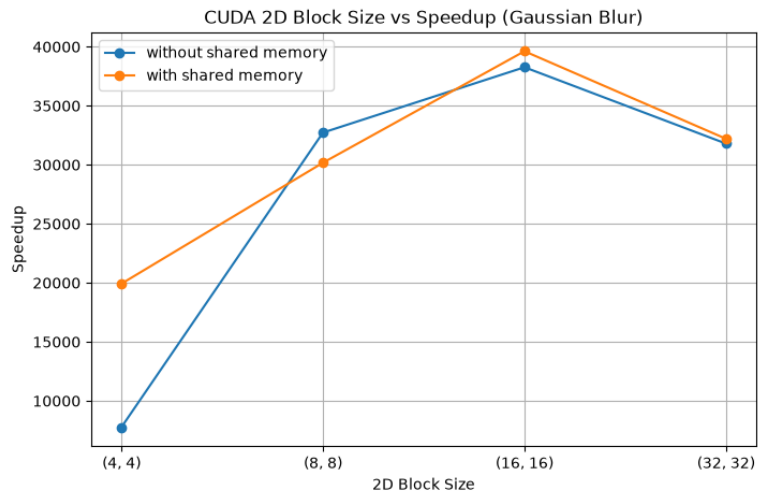


Figure 1: 2D block size vs speedup, with and without shared memory.